Reporte

Nombre: Cepeda Macías Raúl Emilio García Najera Eathan Omar Contreras Elizondo Luis Adrián	**Matrícula**: 2145193 2144814 2145018
Nombre del curso: Componentes de Lógica de Negocio	**Nombre del profesor**: Miguel Ángel Gómez Marroquín
Modulo Netacad: 4	**Actividad**: Actividad fundamental 4 **Link de Github:** https://github.com/rulocm07/Actividad-fundamental-4.git
Fecha: 28/10/2026	

Bibliografia (Formato APA)

Evans, E. (2003). Domain-Driven Design: Tackling Complexity in the Heart of Software. Addison-Wesley.

Fowler, M. (2002). Patterns of Enterprise Application Architecture. Addison-Wesley.

Evans, E., & Fowler, M. (2004). Specifications. https://martinfowler.com/apsupp/spec.pdf

Ramírez, S. (s. f.). FastAPI documentation. https://fastapi.tiangolo.com

Python Software Foundation. (s. f.). decimal: Decimal fixed point and floating point arithmetic. https://docs.python.org/3/library/decimal.html

Gómez Marroquín, M. A. (2026). CLN_U4 Manual de alumnos ilustrado y Práctica guiada TechStore [Material de clase]. FIME, UANL.

Objetivo

Construir la lógica principal para administrar los pedidos de TechStore mediante un diseño orientado al dominio (Domain-Driven Design, DDD). Para ello, se representan las entidades y sus reglas, se controlan las transiciones de estado y se aplican condiciones combinadas para determinar descuentos. También se publica la funcionalidad mediante una API REST desarrollada con FastAPI y se verifica el comportamiento con 42 pruebas automatizadas ejecutadas en memoria, sin depender de una base de datos externa.

1. Introducción y arquitectura del dominio

Este trabajo implementa la parte central de las reglas de negocio de TechStore siguiendo los principios de DDD. El modelo utiliza objetos de dominio y agregados para mantener los datos coherentes, establece restricciones que deben cumplirse en cada operación y concentra las condiciones de descuento en el patrón Specification. Además, el pedido cambia de estado siguiendo una secuencia definida y registra eventos de dominio cuando una transición se realiza correctamente. La aplicación se ofrece por medio de una API REST creada con FastAPI y se comprueba con pruebas automatizadas usando pytest.

Los datos de prueba se cargan en memoria desde archivos CSV reproducibles que contienen información de clientes, productos, pedidos, partidas de pedido y escenarios. De esta manera, las reglas principales no dependen de un sistema de almacenamiento ni quedan ligadas al framework web utilizado.

2. Cálculo manual de pedidos

Antes de ejecutar la implementación, se revisaron los archivos pedidos.csv, detalle_pedidos.csv y productos.csv. Con sus datos se calcularon manualmente las cantidades y los importes de tres pedidos de referencia.

Pedido	Cliente	Unidades y productos	Operación manual	Total
O001	C001 — Ana Torres	4 unidades: 2 de SKU001 y 2 de SKU002	(2 × $600.00) + (2 × $450.00)	$2,100.00 MXN
O006	C006 — Pedro Soto	1 unidad de SKU008	1 × $18,000.00	$18,000.00 MXN
O010	C010 — Jorge Luna	2 unidades: 1 de SKU003 y 1 de SKU009	(1 × $120.00) + (1 × $380.00)	$500.00 MXN

Estos cálculos funcionan como referencia para comparar los resultados que produce el sistema al cargar y procesar los pedidos.

3. Punto de control: carga de datos CSV

Una vez implementadas las entidades y los objetos de valor en app/domain.py, se ejecutó el archivo cargar_csv.py. La comprobación mostró que los 16 pedidos incluidos en los datos de prueba se cargaron en memoria con estado inicial DRAFT, conservando las cantidades y los importes esperados.



4. Registro de resultados de los 15 escenarios (E01–E15)

Los quince casos definidos en datos/escenarios.csv se ejecutaron usando instancias nuevas obtenidas con load_fixture(). La tabla resume la operación realizada, el resultado observado y la razón de negocio que explica cada respuesta.

Caso	Pedido	Estado inicial	Operación	Resultado	Justificación
E01	O001	DRAFT	Evaluar descuento	10 %	Cliente VIP; total de $2,100 y 4 unidades. Cumple los requisitos del descuento del 10 % y no tiene promoción.
E02	O002	DRAFT	Evaluar descuento	0 %	Cliente no VIP y total de $1,050. No alcanza los criterios de ninguna de las reglas de descuento.
E03	O003	DRAFT	Confirmar	Rechazado	El cliente C003 está inactivo. La confirmación exige que el cliente se encuentre activo; no se registra evento.
E04	O005	DRAFT	Evaluar descuento	0 %	El pedido tiene una promoción activa, por lo que queda excluido mediante la especificación de ausencia de promoción.
E05	O006	DRAFT	Evaluar descuento	5 %	Aunque el cliente no es VIP, el importe supera $5,000 y no hay

					promoción activa; se aplica la regla alternativa.
E06	O007	DRAFT	Confirmar	Rechazado	El pedido no contiene unidades. Para confirmarse necesita incluir al menos un artículo.
E07	O008	DRAFT	Confirmar	Éxito: CONFIRMED	El pedido contiene exactamente 20 unidades de SKU003 y el cliente está activo. Se emite OrderConfirmed.
E08	O009	DRAFT	Agregar 2 unidades de SKU003	Rechazado	El pedido ya suma 19 unidades. Agregar dos llevaría el total a 21, por encima del máximo permitido de 20.
E09	O010	DRAFT	Evaluar descuento	0 %	El importe es exactamente $500. La regla requiere una cantidad estrictamente mayor a $500.
E10	O011	DRAFT	Evaluar descuento	0 %	El cliente es VIP y el total supera $500, pero el pedido solo tiene

					3 unidades; se requieren al menos 4.
E11	O012	DRAFT	Confirmar y enviar sin guía	Envío rechazado	La confirmación se realiza y genera OrderConfirmed, pero el envío no procede porque el número de guía está vacío.
E12	O013	DRAFT	Confirmar dos veces	Segunda confirmación rechazada	La primera confirmación es válida. La siguiente falla porque solo un pedido en DRAFT puede confirmarse.
E13	O014	DRAFT	Confirmar, enviar y cancelar	Cancelación rechazada	El pedido llega a CONFIRMED y después a SHIPPED. No se permite cancelar desde SHIPPED; se conservan los eventos válidos anteriores.
E14	O015	DRAFT	Confirmar, enviar y devolver	Éxito: RETURNED	La secuencia DRAFT → CONFIRMED → SHIPPED → RETURNED es válida y genera los tres eventos correspondientes.

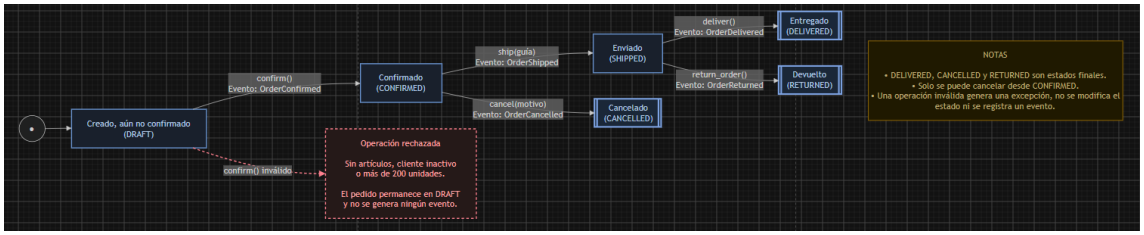
E15	O016	DRAFT	Confirmar, enviar y entregar	Éxito: DELIVERED	La secuencia DRAFT → CONFIRMED → SHIPPED → DELIVERED se completa correctamente y registra sus eventos.

```
(.venv) PS C:\5to semestre\Componentes de logica\Unidad-4-main> python ejecutar_escenarios.py
================================================================================================
(.venv) PS C:\5to semestre\Componentes de logica\Unidad-4-main> & "c:\5to semestre\Componentes de logica\Unidad-4-main\.venv\Scripts\python.exe" "c:/5to semestre/
Componentes de logica/Unidad-4-main/ejecutar_escenarios.py"
================================================================================================
BITÁCORA DE EVALUACIÓN DE ESCENARIOS E01 - E15 | TECHSTORE U4
================================================================================================
------------------------------------------------------------------------------------------------
E01   | O001   | Creado, aún no confirmado (DRAFT) | evaluar_descuento              | Descuento: 10%                  | Sin eventos (lectura)
E02   | O002   | Creado, aún no confirmado (DRAFT) | evaluar_descuento              | Descuento: 0%                   | Sin eventos (lectura)
E03   | O003   | Creado, aún no confirmado (DRAFT) | confirmar                      | RECHAZO (InvalidOrderData)      | Ninguno
E04   | O005   | Creado, aún no confirmado (DRAFT) | evaluar_descuento              | Descuento: 0%                   | Sin eventos (lectura)
E05   | O006   | Creado, aún no confirmado (DRAFT) | evaluar_descuento              | Descuento: 5%                   | Sin eventos (lectura)
E06   | O007   | Creado, aún no confirmado (DRAFT) | confirmar                      | RECHAZO (InvalidOrderData)      | Ninguno
E07   | O008   | Creado, aún no confirmado (DRAFT) | confirmar                      | ÉXITO (Confirmado)              | ['OrderConfirmed']
E08   | O009   | Creado, aún no confirmado (DRAFT) | agregar SKU003 x2              | RECHAZO (OrderItemLimitExceeded)| Ninguno
E09   | O010   | Creado, aún no confirmado (DRAFT) | evaluar_descuento              | Descuento: 0%                   | Sin eventos (lectura)
E10   | O011   | Creado, aún no confirmado (DRAFT) | evaluar_descuento              | Descuento: 0%                   | Sin eventos (lectura)
E11   | O012   | Creado, aún no confirmado (DRAFT) | confirmar y enviar sin guía    | RECHAZO (InvalidOrderData)      | ['OrderConfirmed']
E12   | O013   | Creado, aún no confirmado (DRAFT) | confirmar dos veces            | RECHAZO (InvalidOrderTransition)| ['OrderConfirmed']
E13   | O014   | Creado, aún no confirmado (DRAFT) | confirmar, enviar y cancelar   | RECHAZO (InvalidOrderTransition)| ['OrderConfirmed', 'OrderShipped']
E14   | O015   | Creado, aún no confirmado (DRAFT) | confirmar, enviar y devolver   | ÉXITO (Devuelto)                | ['OrderConfirmed', 'OrderShipped',
'OrderReturned']
E15   | O016   | Creado, aún no confirmado (DRAFT) | confirmar, enviar y entregar   | ÉXITO (Entregado)               | ['OrderConfirmed', 'OrderShipped',
'OrderDelivered']
```

5. Máquina de estados y eventos de dominio

El agregado Order administra su ciclo de vida mediante una máquina de estados finita y predecible. Todo pedido comienza en DRAFT y solo puede avanzar por las transiciones autorizadas: DRAFT a CONFIRMED mediante confirm(); CONFIRMED a SHIPPED mediante ship(); CONFIRMED a CANCELLED mediante cancel(); SHIPPED a DELIVERED mediante deliver(); y SHIPPED a RETURNED mediante return_order(). Los estados DELIVERED, CANCELLED y RETURNED son terminales.

Cuando una operación intenta realizar un cambio no permitido, se genera InvalidOrderTransition. Si los datos no cumplen las reglas, se produce InvalidOrderData. En ambos casos se evita alterar el estado o agregar un evento nuevo. Por ejemplo, un pedido sin artículos o asociado a un cliente inactivo permanece en DRAFT cuando se rechaza su confirmación.



Eventos que registra el sistema

Evento	**Cuándo se genera**
OrderConfirmed	Al cambiar de DRAFT a CONFIRMED, una vez verificados los artículos y la actividad del cliente.
OrderShipped	Al pasar de CONFIRMED a SHIPPED después de validar que existe una guía de envío no vacía.
OrderCancelled	Al pasar de CONFIRMED a CANCELLED con un motivo válido. No se permite cancelar desde DRAFT ni después del envío.
OrderDelivered	Cuando un pedido en SHIPPED pasa al estado DELIVERED.
OrderReturned	Cuando un pedido enviado cambia de SHIPPED a RETURNED.

Los eventos se manejan de forma atómica: primero se revisan las condiciones y la transición; después se actualiza el estado y, finalmente, se agrega el evento al historial order.events. Si ocurre una excepción, el proceso se detiene antes de registrar un cambio inválido. Así, el historial representa únicamente acciones que sí se completaron.

6. Patrón Specification y combinación de reglas

Para evitar que las condiciones comerciales estén repetidas en distintas partes del programa, cada criterio se encapsuló en una especificación reutilizable. Estas reglas pueden combinarse mediante operadores booleanos como AND (&), OR (|) y NOT (~).

Especificación	**Función**
MinimumAmountSpec(Decimal('500'))	Comprueba que el total sea mayor que $500; el límite no se incluye.
VIPCustomerSpec()	Verifica que el cliente esté identificado como VIP.
MinimumUnitsSpec(4)	Valida que el pedido tenga cuatro unidades o más.

ActivePromotionSpec()	Comprueba si el pedido tiene una promoción activa.

Las dos reglas compuestas se definen de la siguiente manera:

Descuento del 10 %: MinimumAmountSpec(500) & VIPCustomerSpec() & MinimumUnitsSpec(4) & ~ActivePromotionSpec().

Descuento del 5 %: MinimumAmountSpec(5000) & ~ActivePromotionSpec().

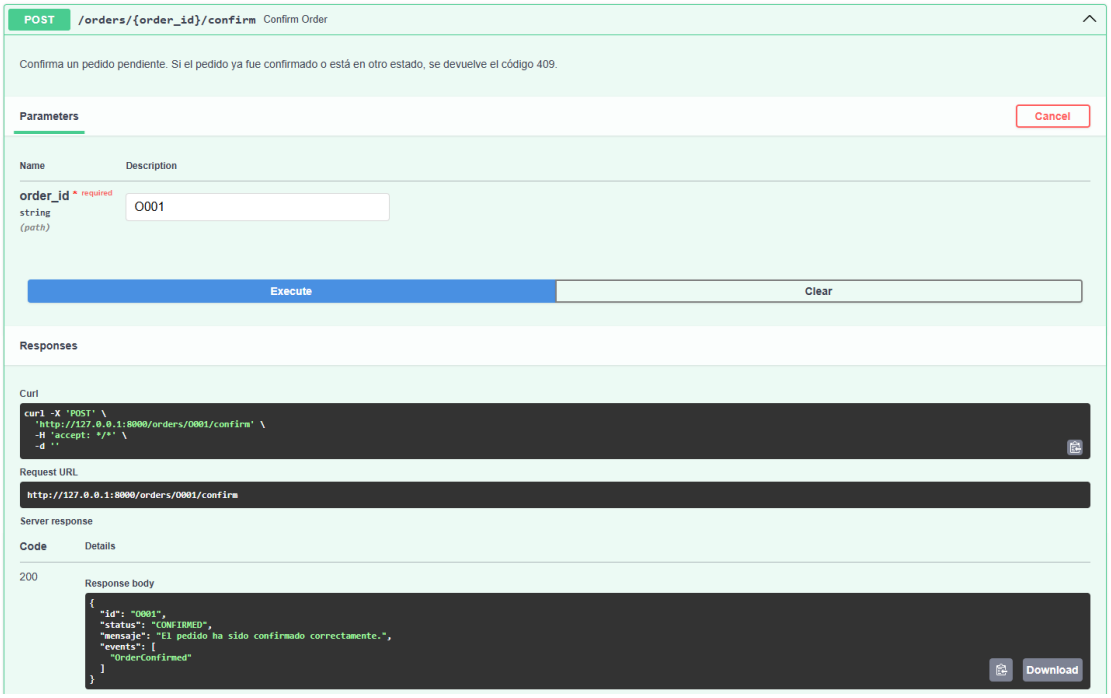
Cuando las dos condiciones pudieran cumplirse a la vez, el sistema da preferencia al descuento del 10 %. La composición permite modificar las políticas comerciales sin dispersar las condiciones por las entidades o los controladores.

7. API REST con FastAPI y códigos HTTP

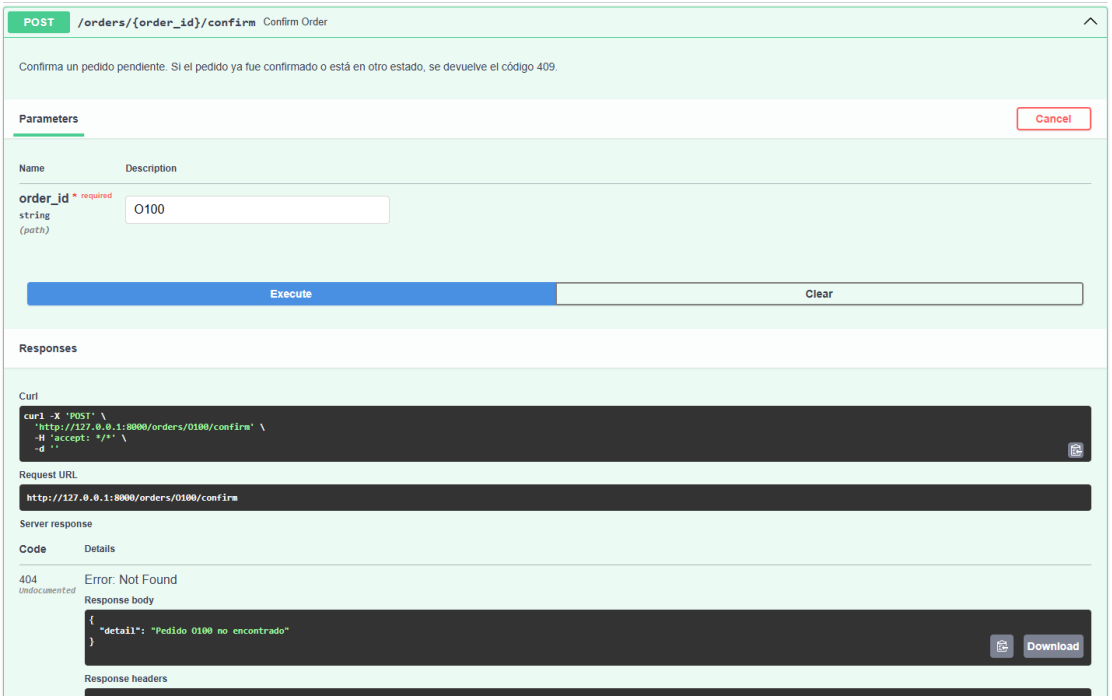
FastAPI funciona como la interfaz de acceso al dominio. Los endpoints llaman a las operaciones del modelo y convierten los errores de negocio en respuestas HTTP apropiadas, de forma que el cliente de la API pueda identificar qué ocurrió.

Código HTTP	**Uso en TechStore**	**Ejemplo**
200 OK	La consulta u operación terminó correctamente.	POST /orders/O001/confirm: el pedido pasa a CONFIRMED.
404 Not Found	No existe el pedido solicitado en memoria.	POST /orders/O999/confirm: pedido no encontrado.
409 Conflict	La operación no corresponde al estado actual del pedido.	Intentar confirmar O013 por segunda vez.
422 Unprocessable Entity	Los datos incumplen una regla o una invariante del dominio.	Confirmar O007 sin artículos o superar el máximo de 20 unidades en O009.

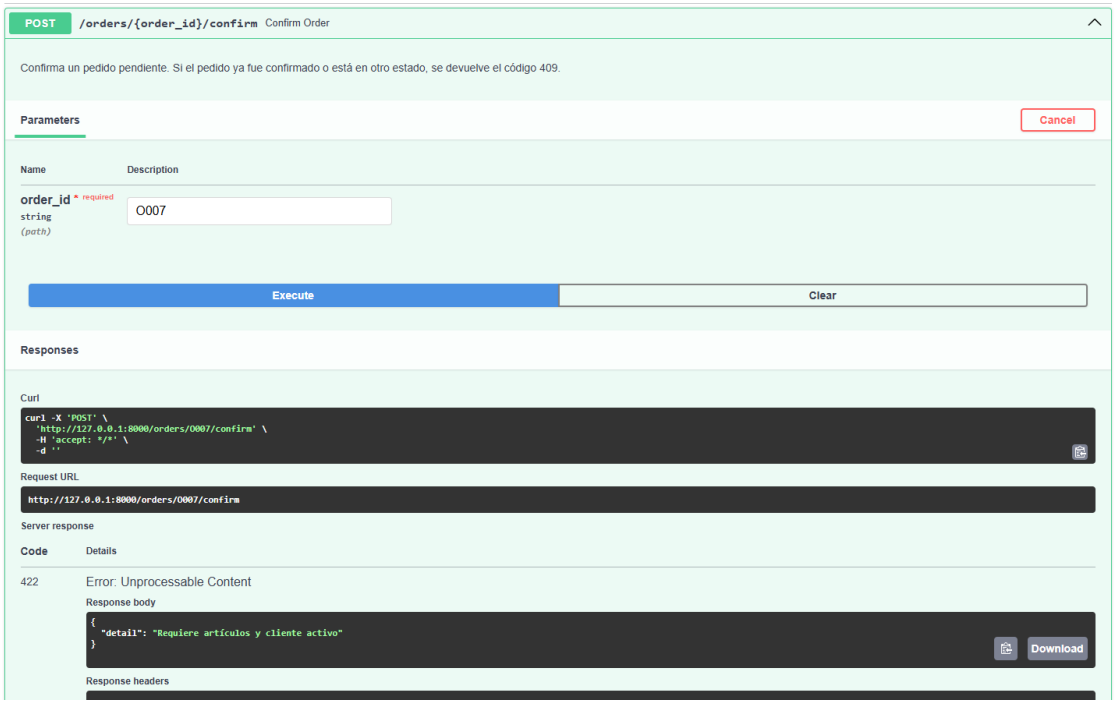
Reporte



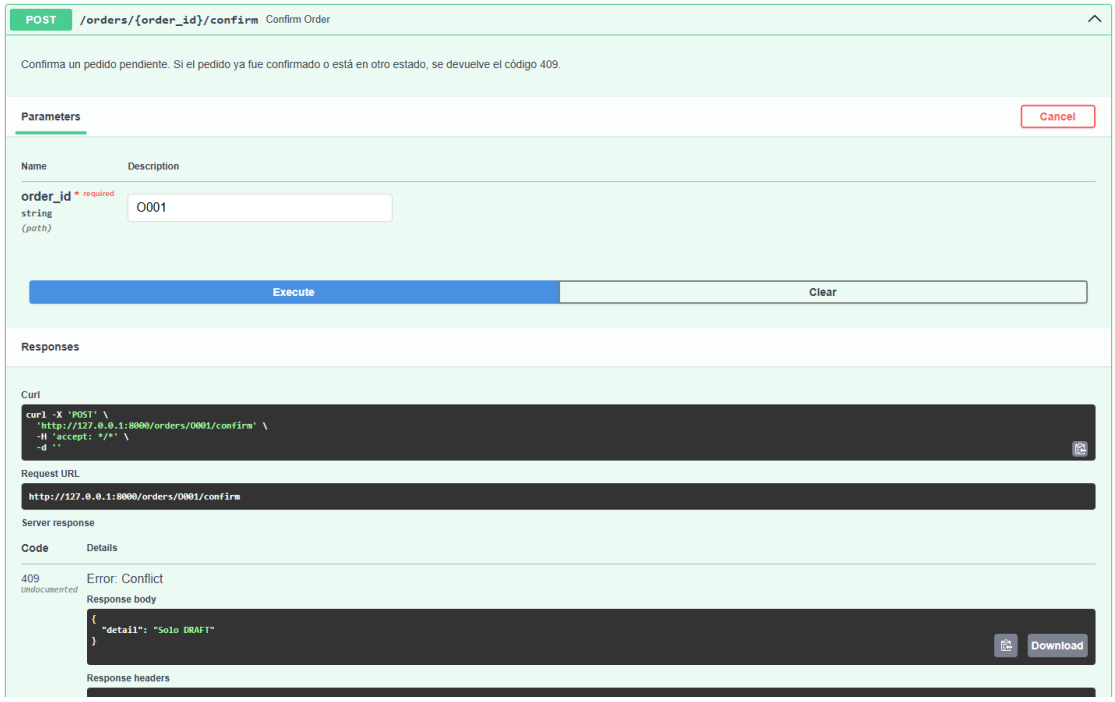
Code 200 la Operación salió correctamente



Code 404 El pedido que buscamos no existe



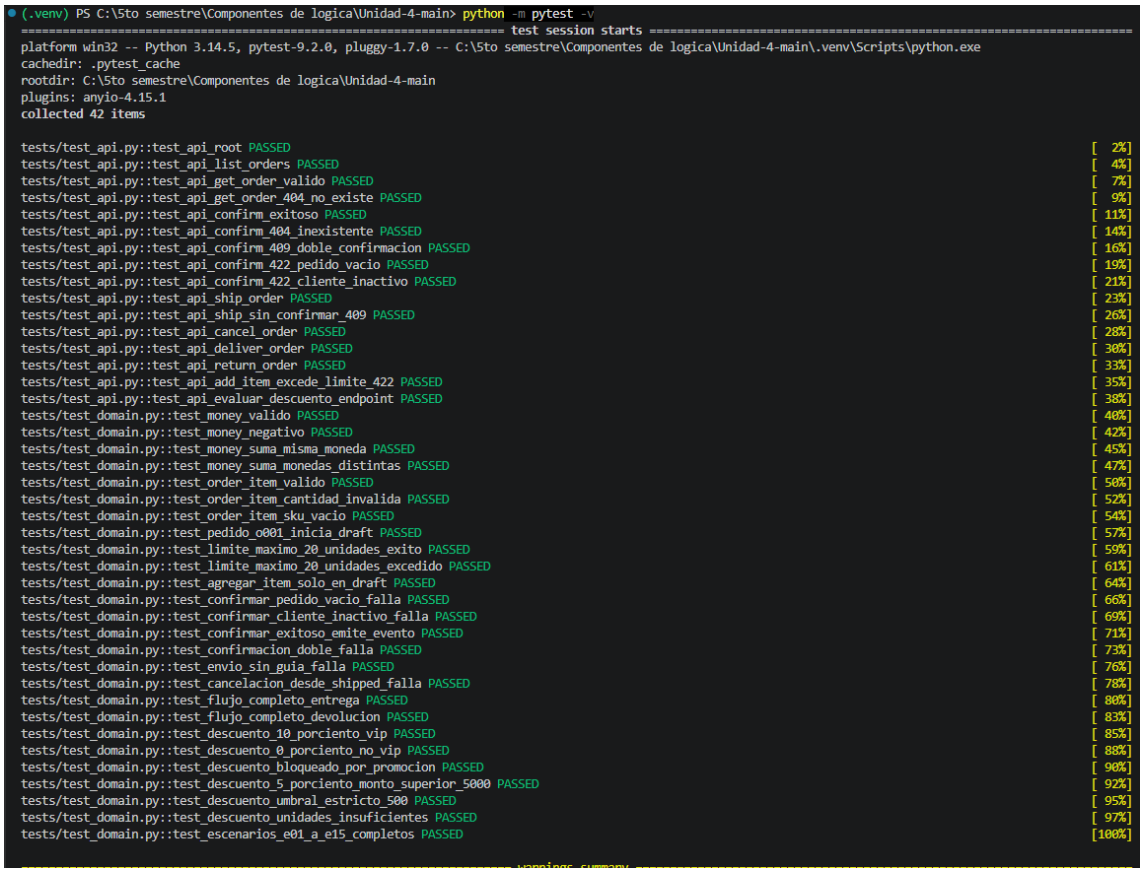
Code 422 Los datos del pedido no cumplen las reglas



Code 409 al confirmar O001 por segunda vez

8. Pruebas automatizadas con pytest

Se preparó un conjunto de 42 pruebas automatizadas distribuidas en tests/test_domain.py y tests/test_api.py. Las pruebas abarcan las reglas del dominio, los casos descritos de E01 a E15 y las respuestas de la API, con el propósito de detectar errores tanto en las operaciones internas como en los endpoints.



Captura de la ejecución de pytest con las 42 pruebas aprobadas

9. Tres problemas identificados, sus soluciones y aprendizajes

Problema 1. Diferencias de precisión entre float y Decimal

Situación: el tipo float puede representar algunos decimales con pequeñas variaciones debido a su codificación binaria. Esto puede afectar importes, descuentos y comparaciones con límites monetarios; por ejemplo, una suma decimal aparentemente sencilla como 0.1 + 0.2 puede no coincidir exactamente con 0.3.

Solución: se creó el objeto de valor inmutable Money usando Decimal. Los importes se inicializan a partir de cadenas decimales exactas, como '600.00', y se validan para impedir cantidades negativas o no finitas.

Aprendizaje: en una aplicación de ventas es importante que los cálculos monetarios mantengan precisión consistente, ya que los errores de redondeo pueden alterar totales y descuentos.

Problema 2. Contar tipos de producto en vez de unidades

Situación: utilizar len(order.items) para comprobar el máximo del pedido contaría los renglones o tipos de producto, no la cantidad real de piezas. Un solo renglón podría representar muchas unidades.

Solución: se definió la propiedad units como la suma de item.quantity para todos los artículos del pedido. Antes de agregar un producto, add_item() comprueba que las unidades existentes más las nuevas no excedan 20.

Aprendizaje: las validaciones deben reflejar lo que la regla comercial realmente limita. En este caso, el límite se refiere a las piezas que se empacan, no al número de productos diferentes.

Problema 3. Registrar eventos antes de confirmar una operación

Situación: si un evento se añadiera antes de terminar las validaciones, una operación rechazada podría dejar un evento en el historial aunque el estado no hubiera cambiado.

Solución: las operaciones siguen tres pasos: validar las precondiciones e invariantes, actualizar el estado y después crear y agregar el evento correspondiente.

Aprendizaje: los eventos de dominio deben documentar hechos reales. Este orden mantiene el historial coherente y evita registrar acciones que el sistema no aceptó.

10. Respuestas a las preguntas del reporte

1. ¿Por qué se utiliza Decimal en vez de float para los precios?

Porque float almacena números de punto flotante en representación binaria y no expresa de forma exacta todos los decimales comunes. En operaciones de venta, estas diferencias pueden acumularse en subtotales, impuestos y descuentos. Decimal permite trabajar con aritmética decimal de mayor control, adecuada para mantener la exactitud requerida en los importes.

2. ¿Qué diferencia existe entre contar renglones y sumar unidades?

Contar los renglones de order.items indica cuántos tipos de producto se incluyeron. Sumar item.quantity calcula la cantidad total de piezas. Por ejemplo, un pedido con un solo renglón que contenga 20 piezas tiene un renglón, pero 20 unidades. Como TechStore permite un máximo de 20 piezas, la validación debe sumar las cantidades.

3. ¿Qué beneficio ofrece el patrón Specification?

Permite separar las reglas de evaluación del resto del modelo y reutilizarlas en distintos componentes. También facilita combinar condiciones simples mediante operadores booleanos para expresar políticas más complejas. Finalmente, cada especificación puede probarse por separado, haciendo más sencillo comprobar su funcionamiento y mantener las reglas.

4. ¿Por qué se agregan los eventos después de una transición válida?

Un evento de dominio describe una acción que realmente ocurrió. Si se registrara antes de validar los datos, el historial podría indicar que un pedido se confirmó o envió cuando la operación fue rechazada. Esto podría desencadenar notificaciones o procesos incorrectos y perjudicar la auditoría del sistema.

5. ¿Qué componentes podrían conservarse si se cambia FastAPI por otra interfaz?

Se pueden mantener las clases del dominio definidas en app/domain.py, como Money, Customer, OrderItem, Order, OrderStatus y DomainEvent, además de las especificaciones y la función evaluate_discount de app/specifications.py. También puede reutilizarse cargar_csv.py. Estos componentes no dependen directamente de FastAPI, por lo que podrían emplearse desde una interfaz de consola, una aplicación de escritorio o un framework web diferente.

11. Lista de comprobación final

Punto revisado	**Evidencia indicada en el reporte**	**Estado**
Ejecución de cargar_csv.py	Carga correcta de los 16 pedidos.	Cumplido
Pedido O001	Se conserva con 4 unidades y un total de $2,100.00 MXN.	Cumplido
Regla de descuento del 10 %	Se revisaron cliente VIP, total mayor a $500, al menos 4 unidades y ausencia de promoción.	Cumplido

Regla de descuento del 5 %	Se revisaron total mayor a $5,000 y ausencia de promoción.	Cumplido
Operaciones inválidas	Los casos indicados no producen cambios de estado ni eventos inválidos.	Cumplido
Respuestas de la API	Se verificaron los códigos 404, 409 y 422 en los casos correspondientes.	Cumplido
Ejecución de pytest	42 pruebas aprobadas, según la evidencia del reporte original.	Cumplido
Repositorio	Se indica que incluye README, .gitignore, código y evidencias.	Cumplido

Conclusiones

La implementación de DDD en TechStore permitió mantener las reglas comerciales separadas de la infraestructura. El uso de entidades y objetos de valor ayuda a conservar la coherencia de la información y permite que la lógica principal no dependa de FastAPI ni de un mecanismo de almacenamiento específico.

El patrón Specification hizo posible organizar y combinar las condiciones de descuento de una forma clara y reutilizable. A su vez, la máquina de estados y los eventos de dominio establecieron un flujo controlado para el pedido: las operaciones solo modifican su estado cuando cumplen las reglas, y los eventos se agregan después de una transición exitosa.

Por último, el tratamiento preciso de los importes mediante Decimal y la validación de las unidades físicas ayudaron a proteger las reglas monetarias y de inventario. Las 42 pruebas automatizadas reportadas aportan evidencia de que los principales escenarios y restricciones fueron comprobados. La separación del dominio también facilita que sus componentes puedan aprovecharse en futuras interfaces o frameworks.